



FOR JAVA DEVELOPERS MOVING TO NESTJS

Debit. No credit.

Spring Boot to NestJS, episode 15: transactions and idempotency

A transaction only protects work that joins it. A check before an insert does not enforce uniqueness.

In TypeORM, only what goes through the transaction's manager is in the transaction. A debit saved through the injected repository survived the rollback.

In Spring, injected repositories join the surrounding transaction, but a `@Transactional` method called through this never crosses the proxy: both rows survived.

Check then insert duplicated on both stacks. A unique constraint allows at most one row; resolve the loser after the failed insert has rolled back.

In scope

- `@Transactional` against `ds.transaction(manager)`
- Self-invocation
- The idempotency race and the unique constraint
- The outbox table, inside one transaction

Not in scope

- Publishing the outbox to Kafka: the Kafka episode
- Propagation modes beyond the default

One machine, ten concurrent requests. The race counts vary from run to run; the row counts under a constraint do not.

01 The measurements

TypeORM, DEBIT VIA INJECTED REPO, THEN THROW	1 row left: ["debit"]
TypeORM, REPO FROM THE MANAGER	0 rows left
SPRING, @Transactional CALLED THROUGH this	2 rows left
CHECK THEN INSERT, NO CONSTRAINT	duplicates on both; count varies by run
UNIQUE CONSTRAINT	1 row; the rest a unique violation (23505)
LOSER TRANSLATED AFTER ROLLBACK	all 10 callers, the same payment id
CATCH INSIDE THE SAME TRANSACTION	TypeORM 25P02; Spring Hibernate session failure
SAME KEY, OTHER AMOUNT (TypeORM)	5 x 409 conflict, 1 row
SERIALIZABLE, NO CONSTRAINT	1 row, 9 x 40001; with retry, 1 row
OUTBOX, EVENT VIA INJECTED REPO	payment 0, outbox 1

Ten concurrent requests with the same idempotency key, the same Postgres 18.



02 Everything through the manager

The callback's manager is the transaction. Take repositories from it.

probes.ts, the fix

```
await ds.transaction(async (m) => {
  const repo = m.getRepository(LedgerEntry);
  await repo.save({ note: 'debit' });
  await repo.save({ note: 'credit' });
  throw new Error('boom');
});
```

Ep14LedgerService.java: through this, no transaction

```
public void postFromInside() {
  this.post();
}
```

debit via injected repository: 1 row left; repository from the manager: 0 rows left

Run on a real machine, 2026-09-30, by scripts/verify-transactions.sh in the course repository.

03 Let the database decide, then translate

A unique constraint on the key allows at most one row. Catch the unique violation outside the failed transaction and return the existing payment. Compare the request too: the same key with a different amount is a conflict, not a replay.

probes.ts

```
try {
  return (await repo.save({ idempotencyKey: 'pay-42' })).id;
} catch (e) {
  if ((e as { code?: string }).code !== '23505') throw e;
  return (await repo.findOneByOrFail({ idempotencyKey: 'pay-42' })).id;
}
```

10 concurrent requests, one key: all 10 got the same payment id, 1 row, on both stacks

Run on a real machine, 2026-09-30, by scripts/verify-transactions.sh in the course repository.

Before you ship transactions and idempotency in NestJS

1. Inside `ds.transaction`, use only the manager and its repositories
2. In Spring, never rely on `@Transactional` through this
3. Put a unique constraint on the idempotency key
4. Resolve the duplicate after the failed insert has rolled back
5. Return a conflict when the same key carries a different request
6. Write the outbox row in the same transaction as the change

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 x https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: <code>scripts/verify-transactions.sh</code> https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	TypeORM transactions https://typeorm.io

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.